



Desarrollo App Móviles II

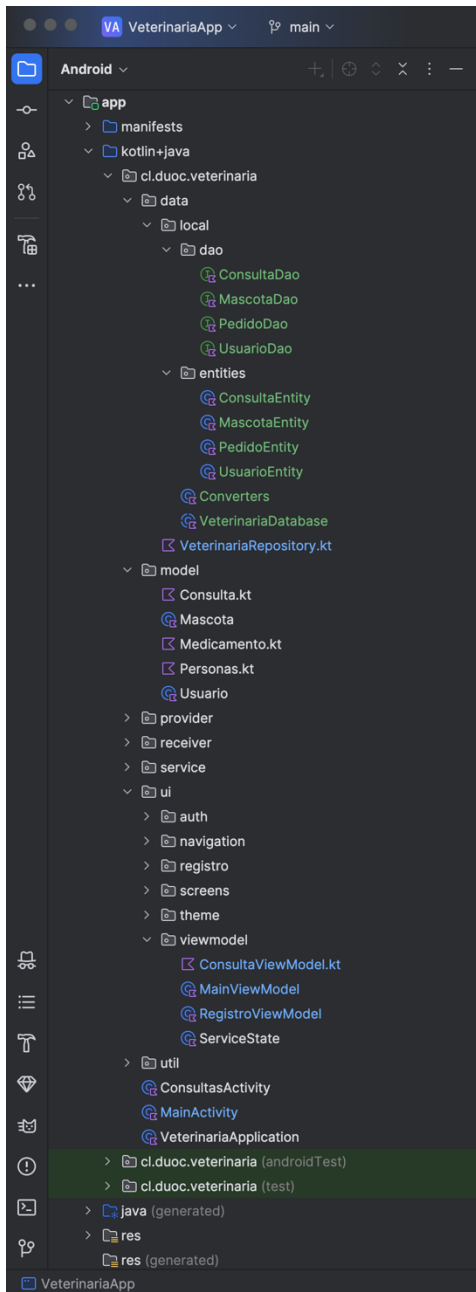
Sumativa N°1

Índice

1.Arquitectura y estructura del proyecto.	3
2. Persistencia de Datos Robusta (Room).....	5
3. Interfaz de usuario moderna y accesibilidad.....	10
4. Lógica de negocio y privacidad (UX).....	13
5. Funcionalidad de farmacia independiente.....	15
6. Servicios del sistema (Notificaciones y compartir).....	17

1.Arquitectura y estructura del proyecto.

Se implementó una arquitectura MVVM (Model-View-ViewModel) para separar la lógica de negocio de la interfaz de usuario. El uso de un NavGraph centralizado asegura una navegación fluida y escalable entre pantallas, permitiendo el paso de parámetros y el control de estados de forma eficiente.



En la imagen se observa la organización del código fuente siguiendo el patrón de arquitectura MVVM (Model-View-ViewModel) y principios de Clean Architecture. El proyecto ha sido estructurado en paquetes modulares para garantizar la escalabilidad y el mantenimiento del código:

- Paquete data/local: Contiene la lógica de persistencia robusta solicitada. Se observan los DAOs (Data Access Objects) y las Entities para las cuatro tablas de la base de datos Room (Consulta, Mascota, Pedido y Usuario), junto con el archivo central VeterinariaDatabase.
- Paquete model: Define los modelos de datos de dominio, manteniendo la lógica de negocio separada de la infraestructura.
- Paquete ui: Organizado por funciones (auth, registro, screens), lo que facilita la navegación y el manejo de la interfaz de usuario.
- Paquete viewmodel: Actúa como puente entre los datos y la interfaz, gestionando el estado de la aplicación de forma reactiva y permitiendo que la lógica de presentación esté totalmente desacoplada de la UI (Cumpliendo el Paso 4 de la pauta).
- VeterinariaRepository.kt: Implementa el Patrón Repositorio, centralizando todas las fuentes de datos (Room y SharedPreferences) en un único punto de acceso para los ViewModels.

```

38 NavHost(
39     navController = navController,
40     startDestination = "bienvenida_screen",
41     enterTransition = { fadeIn() },
42     exitTransition = { fadeOut() }
43 ) {
44     composable(route = "bienvenida_screen") {
45         val mainViewModel: MainViewModel = viewModel()
46         val nombreUsuario = loginViewModel.currentUser?.nombreUsuario ?: "Invitado"
47
48         LaunchedEffect(key1 = nombreUsuario) {
49             mainViewModel.setCurrentUser(nombreUsuario)
50         }
51
52         BienvenidaScreen(
53             onNavigateToNext = { navController.navigate(route = "dueno_screen") },
54             onNavigateToRegistro = { navController.navigate(route = "registro_screen") },
55             onNavigateToListado = { navController.navigate(route = "listado_screen") },
56             onNavigateToPedidos = { navController.navigate(route = "farmacia_solo_screen") },
57             onNavigateToMisAtenciones = { navController.navigate(route = "mis_atenciones_screen") },
58             onNavigateToAgenda = { navController.navigate(route = "agenda_screen") },
59             viewModel = mainViewModel,
60             loginViewModel = loginViewModel
61         )
62     }
63     composable(route = "agenda_screen") {
64         val consultaViewModel: ConsultaViewModel = viewModel()
65         AgendaScreen(
66             duenoNombre = loginViewModel.currentUser?.nombreUsuario ?: "Invitado",
67             onBack = { navController.popBackStack() },
68             viewModel = consultaViewModel
69         )
70     }
71     composable(route = "farmacia_solo_screen") {
72         val nombreUsuario = loginViewModel.currentUser?.nombreUsuario ?: "Invitado"
73         // Limpiamos pero PRECARGAMOS el nombre del usuario logueado
74         LaunchedEffect(key1 = Unit) {
75             registroViewModel.clearData()
76             registroViewModel.updateDatosDueno(nombre = nombreUsuario)
77         }
78         PedidosScreen(
79             viewModel = registroViewModel,
80             onNextClicked = { navController.navigate(route = "resumen_screen") },
81             onBack = { navController.popBackStack() }
82         )
83     }
84     composable(route = "dueno_screen") {
85         DuenoScreen(viewModel = registroViewModel, loginViewModel = loginViewModel, onNextClicked = { navController.navigate(route = "mascota_screen") })
86     }
87     composable(route = "mascota_screen") {
88         MascotaScreen(viewModel = registroViewModel, onNextClicked = { navController.navigate(route = "servicio_screen") })
89     }
90     composable(route = "servicio_screen") {
91         ServicioScreen(viewModel = registroViewModel, onNextClicked = { navController.navigate(route = "pedidos_screen") })
92     }
93     composable(route = "pedidos_screen") {
94         PedidosScreen(viewModel = registroViewModel, onNextClicked = { navController.navigate(route = "resumen_screen") }, onBack = { navController.popBackStack() })
95     }
96     composable(route = "resumen_screen") {
97         ResumenScreen(viewModel = registroViewModel, onConfirmClicked = {
98             registroViewModel.clearData()
99             navController.popBackStack(route = "bienvenida_screen", inclusive = true)
100             navController.navigate(route = "bienvenida_screen")
101         })
102     }
103     composable(route = "listado_screen") {
104         val consultaViewModel: ConsultaViewModel = viewModel()
105         ListadoScreen(onBack = { navController.popBackStack() }, onNavigateToRegistro = { navController.navigate(route = "registro_screen") }, viewModel = consultaViewModel)
106     }
107     composable(route = "mis_atenciones_screen") {
108         val consultaViewModel: ConsultaViewModel = viewModel()
109         AtencionesDuenoScreen(duenoNombre = loginViewModel.currentUser?.nombreUsuario ?: "Invitado", onBack = { navController.popBackStack() }, viewModel = consultaViewModel)
110     }
111 }
112 }
113 }
114 }

```

En estas capturas se muestra la implementación del motor de navegación centralizado utilizando Jetpack Compose Navigation. El NavHost actúa como el orquestador de la aplicación, definiendo las rutas, las transiciones y el paso de datos entre pantallas:

- Sincronización de Sesión Activa: Se observa el uso de LaunchedEffect líneas (48-50) para inyectar el nombre del usuario logueado en el MainViewModel. Esto es fundamental para la seguridad y privacidad, ya que permite que la base de datos filtre automáticamente la información, mostrando a cada usuario solo sus propias mascotas y citas.

- Flujo de Negocio Inteligente: Se

implementaron rutas específicas para separar la Compra Directa en Farmacia (farmacia_solo_screen) del flujo de Registro Médico. Como se observa en el código, al entrar a la farmacia se realiza una limpieza de datos médicos previos (clearData) pero se mantiene la identidad del usuario para vincular la compra a su historial.

- Ciclo de Vida de los Datos: En la pantalla de resumen (líneas 96-101), se observa la lógica de finalización donde, tras confirmar la operación, el sistema limpia el estado del ViewModel y reinicia el grafo de navegación, garantizando que no existan fugas de datos entre registros.

- Navegación Desacoplada: El uso de lambdas (onNavigateToAgenda, onNextClicked) permite que las pantallas de la interfaz de usuario sean totalmente independientes de la lógica de navegación, facilitando el mantenimiento y cumplimiento del patrón MVVM.

2. Persistencia de Datos Robusta (Room)

Para cumplir con el requisito de persistencia sin conexión, se desarrolló una base de datos local con Room. Se diseñaron 4 tablas relacionales que permiten gestionar de forma independiente a los usuarios, sus mascotas, la agenda médica y el historial de compras en farmacia, garantizando la integridad de los datos.

Captura desde Database inspector antes de agregar nuevo usuario, mascota, consulta y pedido en farmacia.

Database Inspector

Network Inspector

Background Task Inspector

Databases

usuarios

mascotas

consultas

Live updates

		idConsulta	mascotaNombre	duenoNombre	descripcion	fechaHora	veterinario	costo	estado	
✓	veterinaria_database									
>	consultas	1	AGENDA-8122	Ron	Illiana	Atención de Vacunación	19/01/2026 09:00	Dr. González	30000.0	Pendiente
>	mascotas	2	AGENDA-9339	Guisepe	Illiana	Atención de Control Sano	19/01/2026 09:00	Dr. Pérez	30000.0	Pendiente
>	pedidos	3	AGENDA-2172	Niacina	Illiana	Atención de Urgencia	19/01/2026 09:00	Dr. González	75000.0	Pendiente
>	room_master_table	4	AGENDA-5211	Humi	Illiana	Atención de Control Sano	19/01/2026 09:00	Dr. Pérez	30000.0	Pendiente
>	usuarios	5	AGENDA-6769	Broco	Emanuel	Atención de Cirugía Menor	19/01/2026 09:00	Dr. Soto	90000.0	Pendiente
		6	AGENDA-3299	tullo	Emanuel	Atención de Control Sano	19/01/2026 09:00	Dr. Soto	30000.0	Pendiente
		7	AGENDA-4748	Pelusa	Pablo	Atención de Control Sano	19/01/2026 09:00	Dr. González	30000.0	Pendiente
		8	AGENDA-9313	Cromo	Pablo	Atención de Vacunación	19/01/2026 09:00	Dr. Pérez	30000.0	Pendiente
		9	AGENDA-2339	Patana	Illiana	Atención de Control Sano	19/01/2026 09:00	Dr. Soto	30000.0	Pendiente
		10	AGENDA-2541	Peluchin	Armando	Atención de Control Sano	19/01/2026 09:00	Dr. González	30000.0	Pendiente

Databases

usuarios

mascotas

Live updates

		id	nombre	especie	edad	pesoKg	ultimaVacunacion	nombreDueno	
✓	veterinaria_database								
>	consultas	1	1	Ron	Perro	17	20.0	2026-01-16	Illiana
>	mascotas	2	2	Guisepe	Pez	3	0.4	2026-01-16	Illiana
>	pedidos	3	3	Niacina	Gato	6	11.0	2026-01-16	Illiana
>	room_master_table	4	4	Humi	Huron	1	1.8	2026-01-16	Illiana
>	usuarios	5	5	Broco	Perro	9	12.0	2026-01-16	Emanuel
		6	6	tullo	Hamster	1	0.8	2026-01-16	Emanuel
		7	7	Pelusa	Cabra	5	22.0	2026-01-16	Pablo
		8	8	Cromo	Perro	6	13.0	2026-01-16	Pablo
		9	9	Patana	Iguana	4	2.0	2026-01-16	Illiana
		10	10	Peluchin	Perro	18	17.0	2026-01-18	Armando

Databases

usuarios

mascotas

consultas

pedidos

Live updates

		id	duenoNombre	items	total	fecha	esCompraDirecta
✓	veterinaria_database						
>	consultas	1	1	Antinflamatorio Premium x1	20000.0	16/01 23:01	0
>	mascotas	2	2	Vitaminas Caninas x1	10800.0	16/01 23:04	1
>	pedidos	3	3	Antibiótico Genérico x1	15000.0	16/01 23:12	0
>	room_master_table	4	4	Vitaminas Caninas x1, Antinflamatorio Premium x1, Analgésico Básico x1	38800.0	16/01 23:18	0
>	usuarios	5	5	Antibiótico Genérico x1, Antinflamatorio Premium x1	35000.0	16/01 23:26	0
		6	6	Analgésico Básico x1	8000.0	16/01 23:27	1
		7	7	Antibiótico Genérico x1	15000.0	16/01 23:30	0
		8	8	Analgésico Básico x1	8000.0	18/01 17:22	0

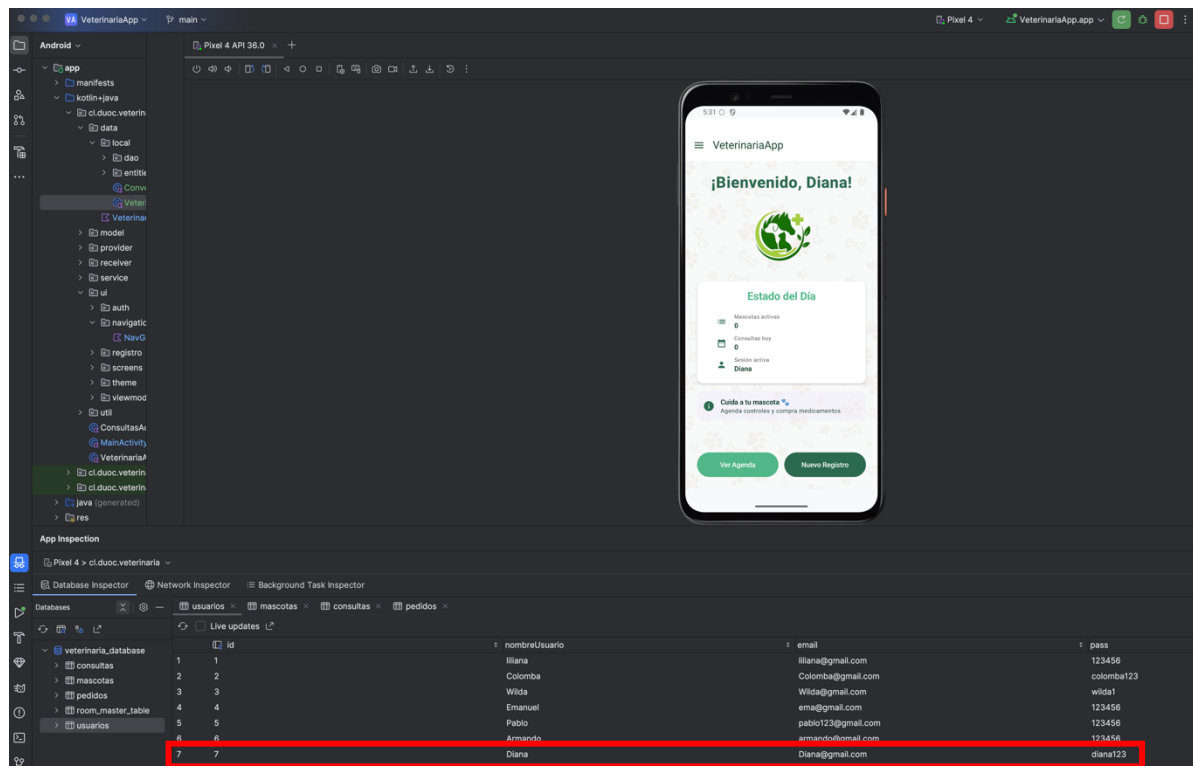
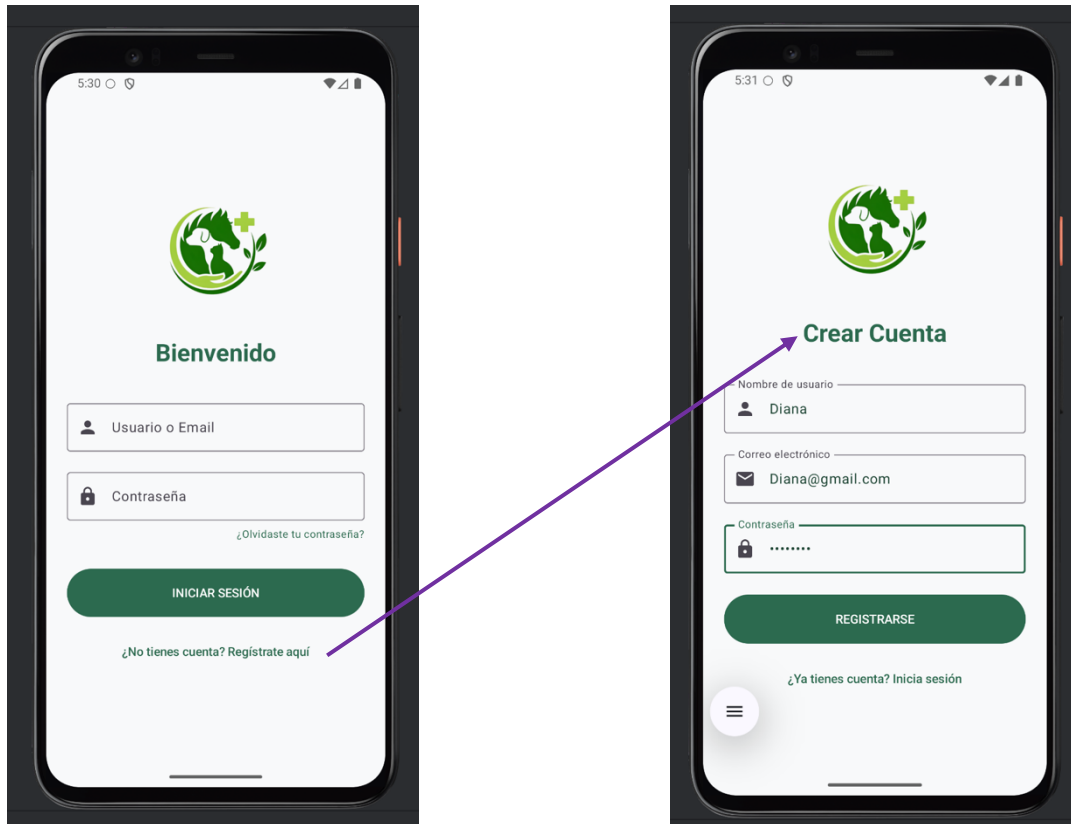
Databases

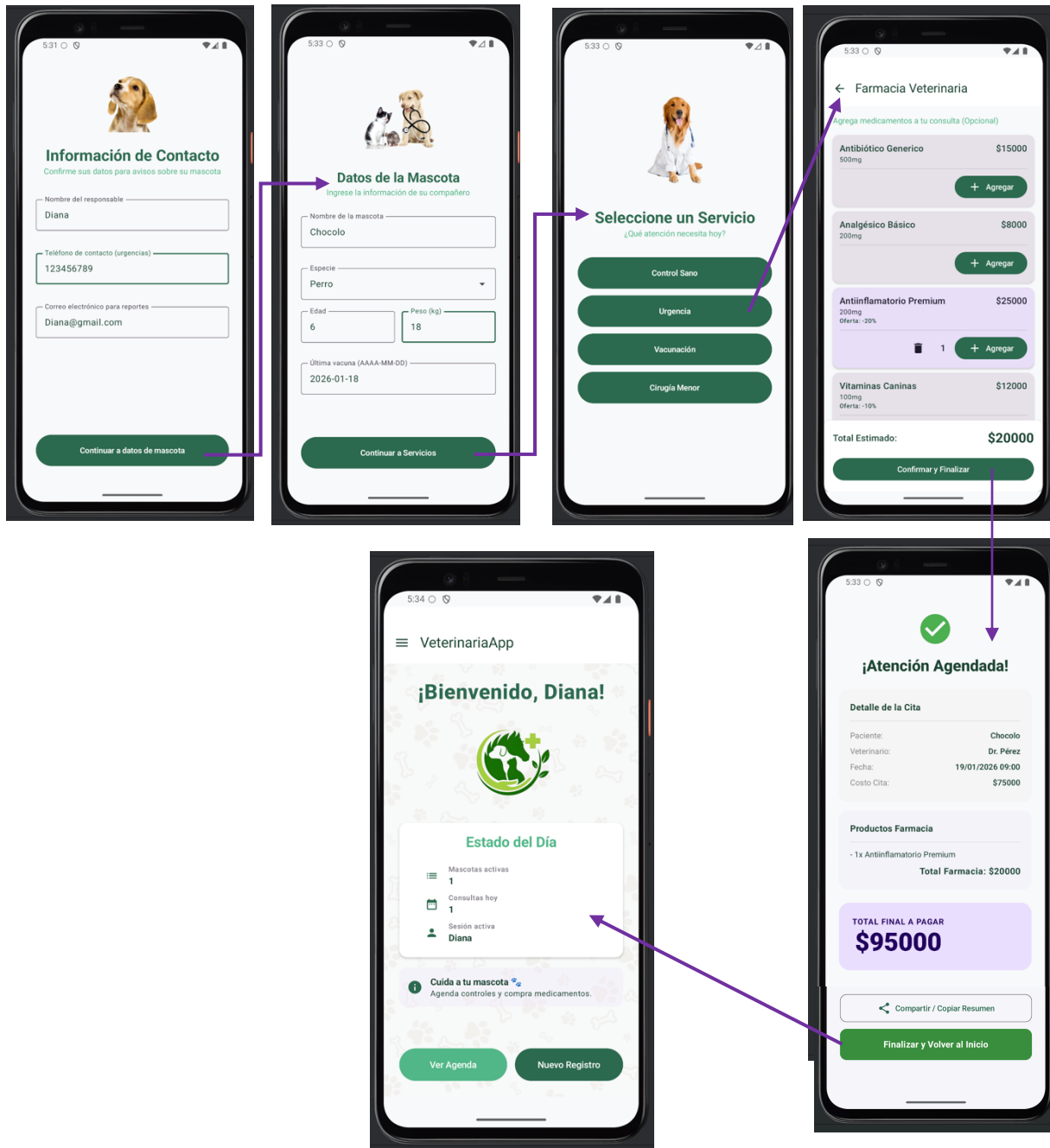
usuarios

Live updates

		id	nombreUsuario	email	pass	
✓	veterinaria_database					
>	consultas	1	1	Illiana	illiana@gmail.com	123456
>	mascotas	2	2	Colomba	colomba@gmail.com	colomba123
>	pedidos	3	3	Wilda	Wilda@gmail.com	wilda1
>	room_master_table	4	4	Emanuel	ema@gmail.com	123456
>	usuarios	5	5	Pablo	pablo123@gmail.com	123456
		6	6	Armando	armando@gmail.com	123456

Ahora se muestran capturas de lo que ocurre al crear nuevo usuario, agregar nueva mascota, nueva consulta y comprar en la farmacia ya sea adentro de la consulta o fuera de esta.





Ahora revisemos nuevamente Database inspector para visualizar los cambios.

	Id	nombre	especie	edad	pesoKg	ultimaVacunacion	nombreDueño
1	1	Ron	Perro	17	20.0	2026-01-16	Illiana
2	2	Guisepppe	Pez	3	0.4	2026-01-16	Illiana
3	3	Niacina	Gato	6	11.0	2026-01-16	Illiana
4	4	Humi	Huron	1	1.8	2026-01-16	Illiana
5	5	Broco	Perro	9	12.0	2026-01-16	Emanuel
6	6	tullo	Hamster	1	0.8	2026-01-16	Emanuel
7	7	Pelusa	Cabra	5	22.0	2026-01-16	Pablo
8	8	Cromo	Perro	6	13.0	2026-01-16	Pablo
9	9	Patana	Iguana	4	2.0	2026-01-16	Illiana
10	10	Peluchin	Perro	18	17.0	2026-01-18	Armando
11	11	Chocolo	Perro	6	18.0	2026-01-18	Diana

Databases

usuarios

mascotas

consultas

pedidos

Live updates

veterinaria_database

consultas

mascotas

pedidos

room_master_table

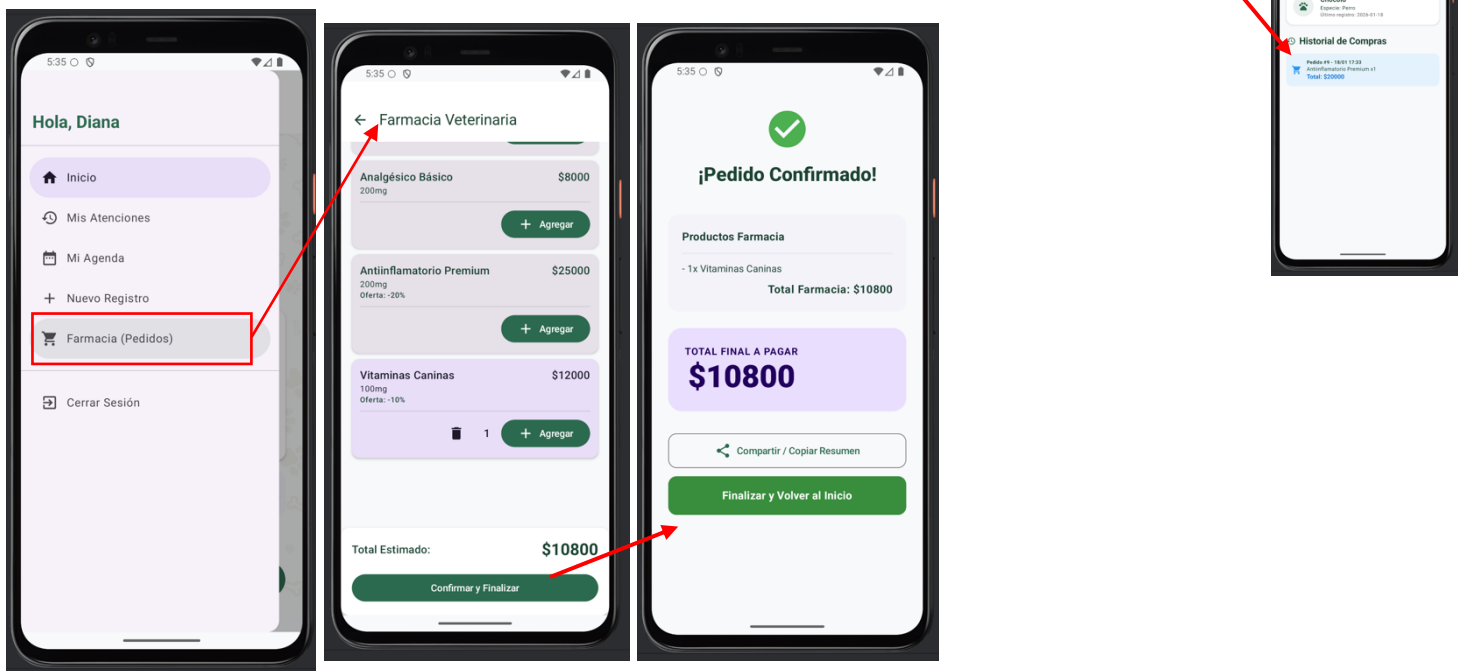
usuarios

	idConsulta	mascotaNombre	duenoNombre	descripcion	fechaHora	veterinario	costo	estado
1	AGENDA-8122	Ron	Illiana	Atención de Vacunación	19/01/2026 09:00	Dr. González	30000.0	Pendiente
2	AGENDA-9339	Guisepe	Illiana	Atención de Control Sano	19/01/2026 09:00	Dr. Pérez	30000.0	Pendiente
3	AGENDA-2172	Niacina	Illiana	Atención de Urgencia	19/01/2026 09:00	Dr. González	75000.0	Pendiente
4	AGENDA-5211	Humi	Illiana	Atención de Control Sano	19/01/2026 09:00	Dr. Pérez	30000.0	Pendiente
5	AGENDA-6769	Broco	Emanuel	Atención de Cirugía Menor	19/01/2026 09:00	Dr. Soto	90000.0	Pendiente
6	AGENDA-3299	tulio	Emanuel	Atención de Control Sano	19/01/2026 09:00	Dr. Soto	30000.0	Pendiente
7	AGENDA-4748	Pelusa	Pablo	Atención de Control Sano	19/01/2026 09:00	Dr. González	30000.0	Pendiente
8	AGENDA-9313	Cromo	Pablo	Atención de Vacunación	19/01/2026 09:00	Dr. Pérez	30000.0	Pendiente
9	AGENDA-2339	Patana	Illiana	Atención de Control Sano	19/01/2026 09:00	Dr. Soto	30000.0	Pendiente
10	AGENDA-2541	Peluchin	Armando	Atención de Control Sano	19/01/2026 09:00	Dr. González	30000.0	Pendiente
11	AGENDA-4912	Chocola	Diana	Atención de Urgencia	19/01/2026 09:00	Dr. Pérez	75000.0	Pendiente

Databases	usuarios	mascotas	consultas	pedidos
veterinaria_database				
consultas				
mascotas				
pedidos				
room_master_table				
usuarios				

id	duenoNombre	Items	total	fecha	esCompraDirecta
1	Illiana	Antiflamatorio Premium x1	20000.0	18/01 23:01	0
2	Illiana	Vitaminas Caninas x1	10800.0	18/01 23:04	1
3	Illiana	Antibiótico Genérico x1	15000.0	16/01 23:12	0
4	Emanuel	Vitaminas Caninas x1, Antiflamatorio Premium x1	38800.0	16/01 23:18	0
5	Pablo	Antibiótico Genérico x1, Antiflamatorio Premium	35000.0	16/01 23:26	0
6	Pablo	Analgésico Básico x1	8000.0	16/01 23:27	1
7	Illiana	Antibiótico Genérico x1	15000.0	16/01 23:30	0
8	Armando	Analgésico Básico x1	8000.0	18/01 17:22	0
9	Diana	Antiflamatorio Premium x1	20000.0	18/01 17:33	0

Farmacia compra directa desde el menu lateral.



Databases	usuarios	mascotas	consultas	pedidos
veterinaria_database				
consultas				
mascotas				
pedidos				
room_master_table				
usuarios				

id	duenoNombre	Items	total	fecha	esCompraDirecta
1	Illiana	Antiflamatorio Premium x1	20000.0	16/01 23:01	0
2	Illiana	Vitaminas Caninas x1	10800.0	16/01 23:04	1
3	Illiana	Antibiótico Genérico x1	15000.0	16/01 23:12	0
4	Emanuel	Vitaminas Caninas x1, Antiflamatorio Premium x1	38800.0	16/01 23:18	0
5	Pablo	Antibiótico Genérico x1, Antiflamatorio Premium	35000.0	16/01 23:26	0
6	Pablo	Analgésico Básico x1	8000.0	16/01 23:27	1
7	Illiana	Antibiótico Genérico x1	15000.0	16/01 23:30	0
8	Armando	Analgésico Básico x1	8000.0	18/01 17:22	0
9	Diana	Antiflamatorio Premium x1	20000.0	18/01 17:33	0
10	Diana	Vitaminas Caninas x1	10800.0	18/01 17:35	1

```

1 package cl.duoc.veterinaria.data.local
2
3 import android.content.Context
4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase
7 import androidx.room.TypeConverters
8 import cl.duoc.veterinaria.data.local.dao.ConsultaDao
9 import cl.duoc.veterinaria.data.local.dao.MascotaDao
10 import cl.duoc.veterinaria.data.local.dao.PedidoDao
11 import cl.duoc.veterinaria.data.local.dao.UsuarioDao
12 import cl.duoc.veterinaria.data.local.entities.ConsultaEntity
13 import cl.duoc.veterinaria.data.local.entities.MascotaEntity
14 import cl.duoc.veterinaria.data.local.entities.PedidoEntity
15 import cl.duoc.veterinaria.data.local.entities.UsuarioEntity
16
17 @Database(entities = [MascotaEntity::class, ConsultaEntity::class, UsuarioEntity::class, PedidoEntity::class], version = 4, exportSchema = false)
18 @TypeConverters(Converters::class)
19 abstract class VeterinariaDatabase : RoomDatabase() {
20     3 Usages 1 Implementation
21     abstract fun mascotaDao(): MascotaDao
22     2 Usages 1 Implementation
23     abstract fun consultaDao(): ConsultaDao
24     4 Usages 1 Implementation
25     abstract fun usuarioDao(): UsuarioDao
26     2 Usages 1 Implementation
27     abstract fun pedidoDao(): PedidoDao
28
29     4 Usages
30     companion object {
31         2 Usages
32         @Volatile
33         private var INSTANCE: VeterinariaDatabase? = null
34
35         1 Usage
36         fun getDatabase(context: Context): VeterinariaDatabase {
37             return INSTANCE ?: synchronized(lock = this) {
38                 val instance = Room.databaseBuilder(
39                     context.applicationContext,
40                     VeterinariaDatabase::class.java,
41                     "veterinaria_database"
42                 )
43                 .fallbackToDestructiveMigration()
44                 .build()
45                 INSTANCE = instance
46                 instance
47             }
48         }
49     }
50 }

```

En esta captura se presenta la implementación central de Room Database, la cual constituye el núcleo de la persistencia de datos local de la aplicación. Los aspectos técnicos más relevantes observados en el código son:

- Definición de Entidades Multi-tabla: Mediante la anotación `@Database`, se integran las cuatro entidades principales del negocio: `MascotaEntity` (gestión de pacientes), `ConsultaEntity` (agenda médica), `UsuarioEntity` (seguridad y sesiones) y `PedidoEntity` (historial de farmacia). Esto demuestra un diseño de base de datos relacional completo y escalable.

- Abstracción de Accesos (DAOs): Se definen los métodos abstractos para acceder a los DAOs, permitiendo una comunicación limpia y segura entre la lógica de negocio y la base de datos SQLite interna.
- Implementación del Patrón Singleton: El uso de un companion object con la anotación `@Volatile` y el método `getDatabase` garantiza que exista una instancia única de la base de datos en toda la aplicación. Esto optimiza el uso de recursos del dispositivo y evita conflictos de escritura de datos.
- Manejo de Tipos Complejos: Se observa la integración de `@TypeConverters(Converters::class)`, permitiendo que la base de datos almacene correctamente objetos que SQLite no soporta nativamente, como las fechas (`LocalDate`) de las consultas y vacunaciones.
- Estrategia de Migración: El uso de `fallbackToDestructiveMigration()` asegura que la aplicación mantenga su estabilidad ante cambios en el esquema de datos durante la fase de prototipado, en producción se usarían migraciones versionadas.

3. Interfaz de Usuario Moderna y accesibilidad

La interfaz utiliza Jetpack Compose y componentes de Material3. Se aplicaron principios de jerarquía visual mediante el uso de Cards con contrastes de color para diferenciar citas futuras de registros pasados. El uso de LazyColumn asegura un rendimiento óptimo (equivalente moderno a RecyclerView) para listas extensas.

```

53 LazyColumn(
54     modifier = Modifier
55         .fillMaxSize()
56         .padding(paddingValues = innerPadding)
57         .padding(horizontal = 16.dp),
58     verticalArrangement = Arrangement.spacedBy(space = 16.dp)
59 ) {
60     item {
61         Spacer(modifier = Modifier.height(height = 16.dp))
62         Text(text = "Hola, $duenoNombre 🐾", style = MaterialTheme.typography.headlineSmall, fontWeight = FontWeight.Bold, color = MaterialTheme.colorScheme.primary)
63         Text(text = "Revisa tus próximas citas y tus mascotas.", style = MaterialTheme.typography.bodyMedium, color = Color(color = 0xFF8BC34A))
64     }
65
66     // SECCIÓN PRÓXIMAS CITAS (Agenda Pura)
67     item {
68         Row(verticalAlignment = Alignment.CenterVertically) {
69             Icon(imageVector = Icons.Default.CalendarMonth, contentDescription = null, modifier = Modifier.size(size = 24.dp), tint = MaterialTheme.colorScheme.primary)
70             Spacer(modifier = Modifier.width(width = 8.dp))
71             Text(text = "Próxima Citas", style = MaterialTheme.typography.titleLarge, fontWeight = FontWeight.Bold)
72         }
73     }
74
75     if (miAgenda.isEmpty()) {
76         item {
77             Card(
78                 modifier = Modifier.fillMaxWidth(),
79                 colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.5f))
80             ) {
81                 Text(text = "No tienes citas agendadas.", modifier = Modifier.padding(padding = 16.dp), style = MaterialTheme.typography.bodyMedium)
82             }
83         }
84     } else {
85         items(items = miAgenda) { consulta ->
86             AgendatItemCard(consulta)
87         }
88     }
89
90     // SECCIÓN MIS MASCOTAS (Información básica)
91     item {
92         Row(verticalAlignment = Alignment.CenterVertically) {
93             Icon(imageVector = Icons.Default.Pets, contentDescription = null, modifier = Modifier.size(size = 24.dp), tint = MaterialTheme.colorScheme.primary)
94             Spacer(modifier = Modifier.width(width = 8.dp))
95             Text(text = "Mis Mascotas", style = MaterialTheme.typography.titleLarge, fontWeight = FontWeight.Bold)
96         }
97     }
98
99     if (misMascotas.isEmpty()) {
100         item {
101             Text(text = "Aún no tienes mascotas registradas.", style = MaterialTheme.typography.bodyMedium, color = MaterialTheme.colorScheme.outline)
102         }
103     } else {
104         items(items = misMascotas) { mascota ->
105             MascotaSimpleCard(mascota)
106         }
107     }
108
109     item { Spacer(modifier = Modifier.height(height = 24.dp)) }
110 }

```

En este fragmento de código se observa la implementación de la pantalla 'Mi Agenda' utilizando el componente LazyColumn. Este componente representa el estándar moderno en Jetpack Compose para el manejo eficiente de listas, ya que renderiza solo los elementos visibles, optimizando el consumo de memoria del dispositivo.

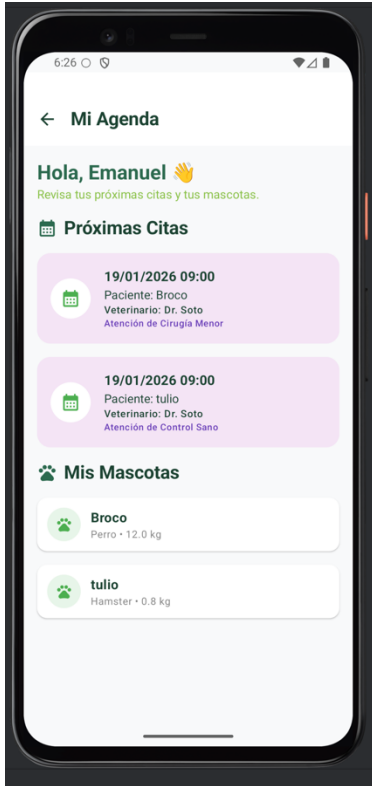
Puntos clave del código:

- Lógica de Estados Vacíos (Empty States): Se

implementan validaciones (if (miAgenda.isEmpty())) para guiar al usuario cuando no existen datos, mejorando la accesibilidad.

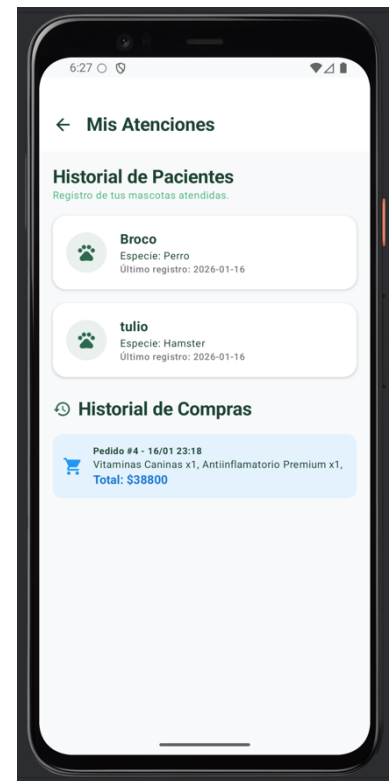
- Jerarquía Visual: Se utiliza una estructura de ítems y encabezados con iconografía descriptiva (CalendarMonth, Pets) para segmentar la información.

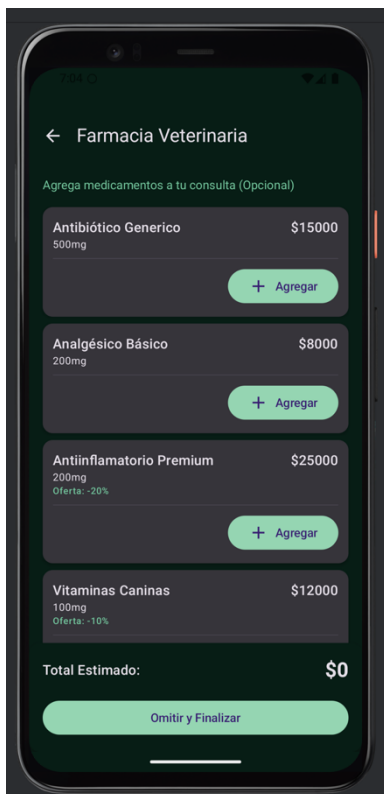
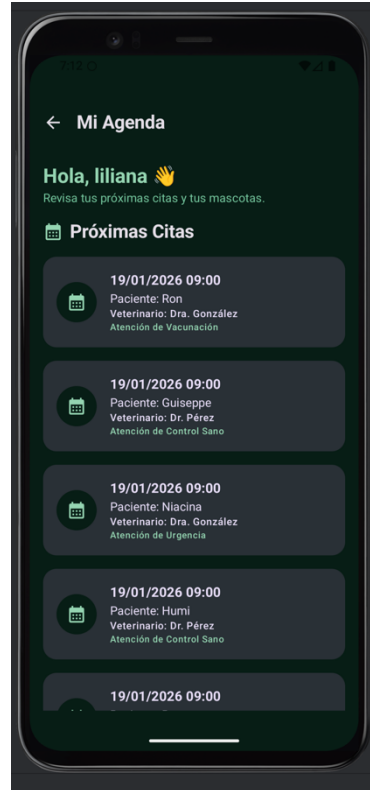
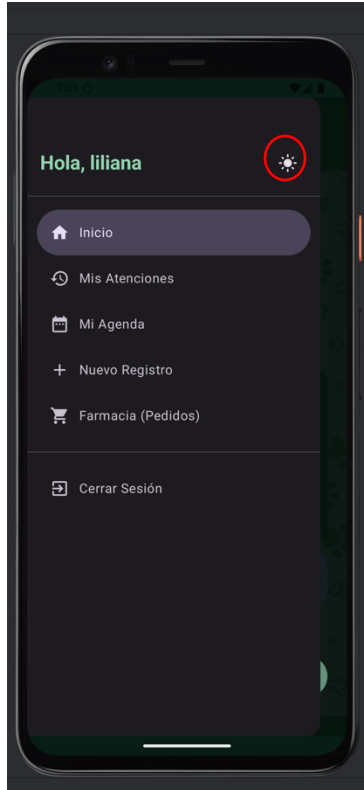
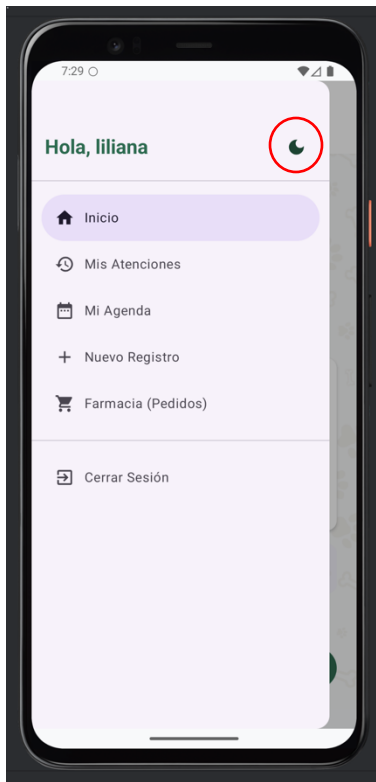
- Modularidad: El código delega el diseño de las tarjetas a componentes especializados (AgendatItemCard, MascotaSimpleCard), siguiendo el principio de responsabilidad única.



La captura de la interfaz de la agenda muestra una experiencia de usuario (UX) personalizada y profesional. Se destaca el saludo dinámico que reconoce al usuario activo y el uso de tarjetas con contrastes de colores suaves (morado y blanco) para diferenciar las secciones. El diseño cumple con las guías de Material Design 3, asegurando una navegación intuitiva donde el usuario puede identificar de un solo vistazo sus próximos compromisos veterinarios y el estado de sus mascotas.

Esta pantalla evidencia la separación lógica entre el historial (pasado) y la agenda (futuro). Se observa un listado limpio de los pacientes atendidos y el historial de compras realizadas. Se aplicaron principios de jerarquía visual y contraste, utilizando fuentes en negrita para los nombres de las mascotas y colores secundarios para los detalles técnicos (especie y fecha), garantizando que la información sea accesible y fácil de leer para cualquier tipo de usuario.





Implementación de Accesibilidad: Control de Tema Dinámico (Modo Oscuro)

Como mejora significativa en accesibilidad y usabilidad, se integró un sistema de Modo Oscuro gestionado por el usuario. Esta funcionalidad permite reducir la fatiga visual en entornos de poca luz y mejora el contraste para usuarios con sensibilidad lumínica. La implementación se realizó mediante un estado global en el MainViewModel, permitiendo una transición fluida y coherente en todas las pantallas de la aplicación sin perder el contexto de la sesión.

4. Lógica de Negocio y Privacidad (UX)

El sistema filtra automáticamente la información de la base de datos basándose en la sesión activa. Esto asegura que un usuario no pueda ver las mascotas ni la agenda de otros, cumpliendo con estándares reales de seguridad y usabilidad.



En la captura del MainViewModel.kt que se presenta en la próxima página, es el núcleo que garantiza la integridad de los datos y la personalización de la interfaz. Su implementación se justifica bajo los siguientes pilares:

1. Segregación de Datos y Privacidad (Control de Acceso): El ViewModel implementa un filtro dinámico basado en el rol del usuario. Mientras que los perfiles de "admin" o "vet" tienen una visión global del sistema, los usuarios regulares están restringidos a ver únicamente sus propios registros (mascotas y consultas). Esto garantiza la privacidad de la información, asegurando que un cliente no pueda visualizar datos de otros dueños, cumpliendo con principios básicos de seguridad de la información.
2. Lógica de Negocio Reactiva: Se utiliza el operador combine de Kotlin Coroutines para unir el flujo de datos del repositorio con el usuario actual, esto permite que las estadísticas de la pantalla de inicio (totalMascotas, totalConsultas) se recalculen automáticamente y en tiempo real si el usuario cambia o

si los datos locales se actualizan, eliminando la necesidad de refrescar la pantalla manualmente (UX fluida).

3. Gestión de Preferencias Visuales (UX): La propiedad `isDarkMode` centraliza el estado visual de la aplicación, al manejar el tema desde un estado global (`StateFlow`), se asegura que la preferencia del usuario sea persistente y consistente en todos los componentes de la aplicación, mejorando la accesibilidad y el confort visual.

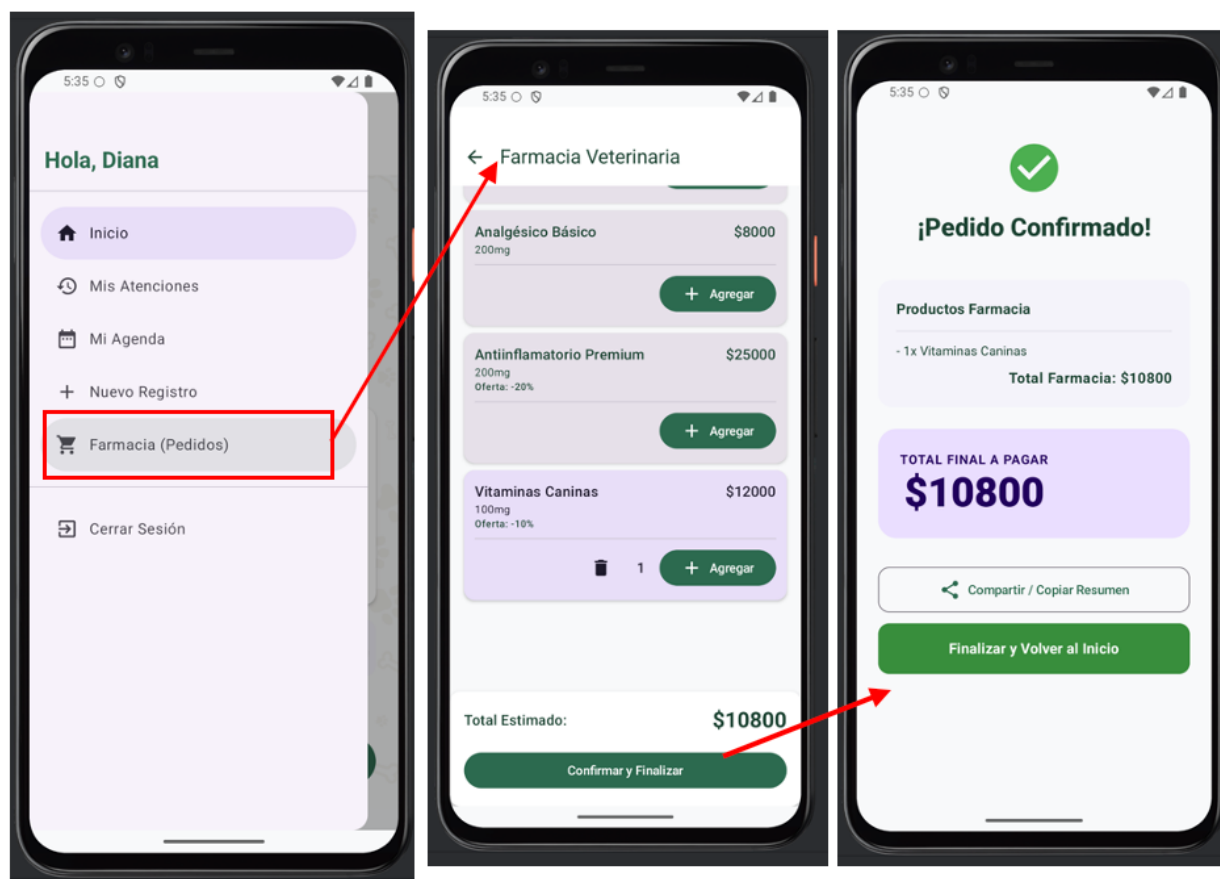
4. Arquitectura Limpia: Al delegar el cálculo de totales al `ViewModel`, la interfaz de usuario (UI) permanece libre de lógica compleja, esto facilita el mantenimiento del código y asegura que la experiencia del usuario sea rápida y eficiente, ya que el procesamiento de datos ocurre fuera del hilo principal de la interfaz.

```
MainViewModel.kt x
3  > import ...
8
9  /**
10 * MainViewModel gestiona los datos mostrados en la pantalla de inicio y el estado global del tema.
11 */
12 class MainViewModel(
13     private val repository: IVeterinariaRepository = VeterinariaRepository
14 ) : ViewModel() {
15
16     3 Usages
17     private val _currentUser = MutableStateFlow(value = "")
18
19     // Estado para el modo oscuro
20     3 Usages
21     private val _isDarkMode = MutableStateFlow(value = false)
22     2 Usages
23     val isDarkMode: StateFlow<Boolean> = _isDarkMode.asStateFlow()
24
25     1 Usage
26     fun toggleDarkMode() {
27         _isDarkMode.value = !_isDarkMode.value
28     }
29
30     /**
31     * Establece el usuario actual para filtrar las estadísticas.
32     */
33     1 Usage
34     fun setCurrentUser(name: String) {
35         _currentUser.value = name
36     }
37
38     // Calcula el total de mascotas solo para el usuario actual
39     1 Usage
40     val totalMascotas: StateFlow<Int> = combine(flow = repository.mascotasLocal, flow2 = _currentUser) { list, user ->
41         if (user.lowercase() == "admin" || user.lowercase() == "vet") {
42             list.size
43         } else {
44             list.count { it.nombreDueno.equals(other = user, ignoreCase = true) }
45         }
46     }.stateIn(viewModelScope, started = SharingStarted.WhileSubscribed(stopTimeoutMillis = 5000), initialValue = 0)
47
48     // Calcula las consultas de hoy solo para el usuario actual
49     1 Usage
50     val totalConsultas: StateFlow<Int> = combine(flow = repository.consultasLocal, flow2 = _currentUser) { list, user ->
51         if (user.lowercase() == "admin" || user.lowercase() == "vet") {
52             list.size
53         } else {
54             list.count { it.duenoNombre.equals(other = user, ignoreCase = true) }
55         }
56     }.stateIn(viewModelScope, started = SharingStarted.WhileSubscribed(stopTimeoutMillis = 5000), initialValue = 0)
57
58 }
```

5. Funcionalidad de Farmacia Independiente

Se dotó a la aplicación de una lógica de negocio real al permitir compras directas en farmacia sin necesidad de una consulta médica. El sistema detecta automáticamente el flujo del usuario y genera un registro persistente en la tabla de pedidos, permitiendo llevar un control histórico de gastos.

Farmacia compra directa desde el menu lateral.



```

117     val nombreCliente = if (currentState.duenoNombre.isBlank()) "Venta Mostrador" else currentState.duenoNombre
118     val esSoloFarmacia = currentState.mascotaNombre.isBlank()
119
120     val nuevoPedido = if (currentState.carrito.isNotEmpty()) {
121         Pedido(Cliente(nombreCliente, email = "", telefono = ""), detalles = currentState.carrito)
122     } else null
123
124     if (nuevoPedido != null) {
125         val itemsTexto = nuevoPedido.detalles.joinToString { "${it.medicamento.nombre} x${it.cantidad}" }
126         val fechaActual = LocalDateTime.now().format(formatter = DateTimeFormatter.ofPattern(pattern = "dd/MM HH:mm"))
127         repository.registrarPedidoRoom(
128             PedidoEntity(duenoNombre = nombreCliente, items = itemsTexto, total = nuevoPedido.total, fecha = fechaActual, esCompraDirecta = esSoloFarmacia)
129         )
130     }
131
132     var consultaFinal: Consulta? = null

```

```

134     if (!esSoloFarmacia) {
135         val (veterinarioAsignado, slots) = AgendaVeterinario.reservarBloque(fechaInicio = AgendaVeterinario.siguienteSlotHabil(Clock.systemDefaultZone()), bloques = 1)
136         val fechaHoraReal = slots.first()
137         val fechaHoraFormateada = AgendaVeterinario.fmt(fechaHoraReal)
138         val servicio = currentState.tipoServicio ?: TipoServicio.CONTROL
139         val costoConsulta = ConsultaService.calcularCostoBase(tipo = servicio, duracionMinutos = 30)
140         val idCita = "AGENDA-" + (1000 ≤ .. ≤ 9999).random()
141
142         repository.registrarAtencion(
143             nombreDueno = nombreCliente,
144             cantidadMascotas = 1,
145             nombreMascota = currentState.mascotaNombre,
146             especieMascota = currentState.mascotaEspecie,
147             tipoServicio = servicio.descripcion,
148             edad = currentState.mascotaEdad.toIntOrNull() ?: 0,
149             peso = currentState.mascotaPeso.toDoubleOrNull() ?: 0.0,
150             consultaId = idCita,
151             fechaHora = fechaHoraFormateada,
152             veterinario = veterinarioAsignado.nombre,
153             costo = costoConsulta
154         )
155
156         consultaFinal = Consulta(
157             idConsulta = idCita,
158             descripcion = "Atención de ${servicio.descripcion}",
159             costoConsulta = costoConsulta,
160             fechaAtencion = fechaHoraReal,
161             veterinarioAsignado = Veterinario(veterinarioAsignado.nombre, especialidad = "")
162         )
163     }

```

La aplicación implementa una arquitectura flexible que permite separar los servicios médicos de los comerciales. La lógica se basa en el estado reactivo del formulario, permitiendo que la farmacia funcione de manera autónoma mediante los siguientes puntos:

1. **Detección Automática de Flujo (Discriminación de Servicio):** El sistema utiliza la variable booleana `esSoloFarmacia`, la cual se activa automáticamente si el usuario no ingresa el nombre de una mascota. Esto permite que el motor de procesamiento (`procesarRegistro`) identifique que no se requiere una reserva de agenda ni asignación de veterinario.
2. **Omisión Inteligente de Lógica Médica:** Como se observa en el código, el bloque de reserva de bloques horarios y cálculos de costo por servicio (`if (!esSoloFarmacia)`) queda totalmente excluido en una compra directa. Esto optimiza los recursos del sistema al no invocar los servicios de `AgendaVeterinario` ni `ConsultaService` innecesariamente.
3. **Persistencia Transaccional Independiente:** Independientemente de si hay una consulta médica, el sistema genera un objeto `Pedido` y lo persiste en la base de datos local (Room) mediante `registrarPedidoRoom`. Si el cliente no está registrado, el sistema asigna automáticamente el nombre

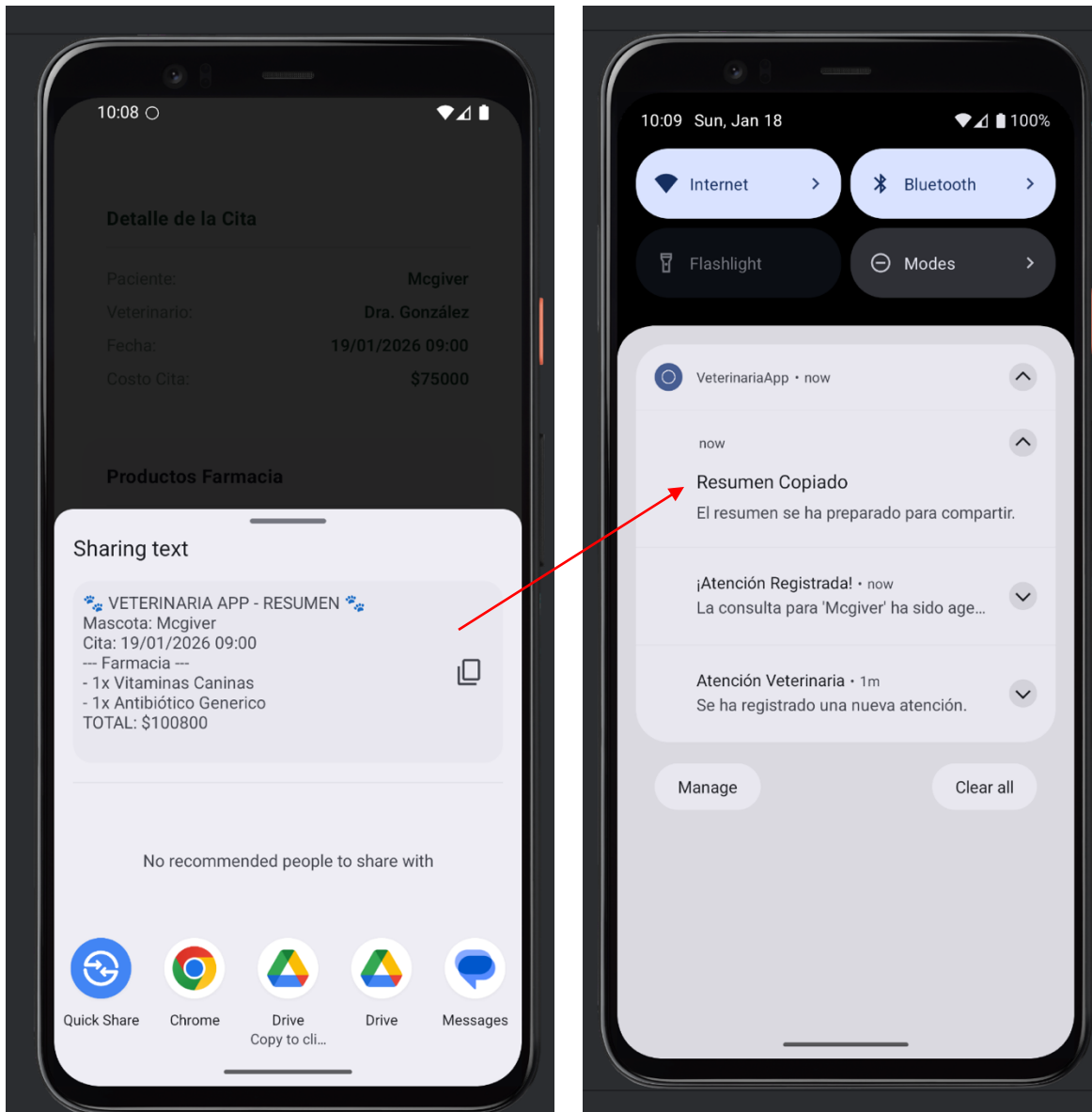
"Venta Mostrador", asegurando que cada transacción de farmacia deje una traza histórica de ítems, totales y fechas.

4.Experiencia de Usuario (UX) Simplificada: Esta lógica permite una "compra rápida", donde un usuario puede simplemente añadir productos al carrito y finalizar la transacción en pocos pasos, recibiendo un resumen de compra detallado sin las restricciones de un flujo clínico tradicional.

6. Servicios del Sistema (Notificaciones y Compartir)

Para mejorar el feedback hacia el usuario, se integró un Service de notificaciones del sistema que informa el éxito de las operaciones. Además, se incluyó una función para Compartir/Copiar Resumen mediante Intents implícitos, facilitando la portabilidad de la información técnica de la atención.





Captura del Intent de Notificación Automática. Se lanza un `LaunchedEffect` para avisar que la atención fue agendada o el pedido fue exitoso.

```

47     LaunchedEffect( key1 = uiState.notificacionAutomaticaMostrada ) {
48         if (uiState.notificacionAutomaticaMostrada) {
49             val titulo = if (esSoloFarmacia) "¡Compra Exitosa!" else "¡Atención Registrada!"
50             val texto = if (esSoloFarmacia) "Tu pedido se ha procesado correctamente." else "La consulta para '${uiState.mascotaNombre}' ha sido agendada."
51             val serviceIntent = Intent( packageContext = context, cls = NotificacionService::class.java ).apply {
52                 putExtra( name = "EXTRA_TITULO", value = titulo )
53                 putExtra( name = "EXTRA_TEXTO", value = texto )
54             }
55             context.startService( serviceIntent )
56             viewModel.onNotificacionMostrada()
57         }
58     }

```

Este bloque lanza un Intent Explícito para iniciar el servicio de notificaciones justo cuando el usuario presiona el botón.

```
178 // --- BOTÓN COMPARTIR CON NOTIFICACIÓN RESTAURADA ---
179 OutlinedButton(
180     onClick = {
181         // Notificación manual al compartir
182         val serviceIntent = Intent(packageContext = context, cls = NotificacionService::class.java).apply {
183             putExtra(name = "EXTRA_TITULO", value = "Resumen Copiado")
184             putExtra(name = "EXTRA_TEXTO", value = "El resumen se ha preparado para compartir.")
185         }
186         context.startService(serviceIntent)
187     }
```

Se implementa un Intent Explícito que invoca al NotificacionService. Al interactuar con el botón de compartir, el sistema envía parámetros adicionales (extras) con el título y mensaje personalizado, notificando al usuario de forma asíncrona que la acción se ha realizado con éxito.

Este bloque lanza un Intent Implícito con la acción ACTION_SEND. Es lo que hace que aparezca la ventana de Android con opciones como WhatsApp, Gmail o Mensajes.

```
202 val sendIntent = Intent().apply {
203     action = Intent.ACTION_SEND
204     putExtra(name = Intent.EXTRA_TEXT, value = resumenTexto)
205     type = "text/plain"
206 }
207 context.startActivity(Intent.createChooser(target = sendIntent, title = "Compartir resumen"))
```

Se utiliza un Intent Implícito con la acción ACTION_SEND para delegar la tarea de comunicación al sistema operativo. El código genera un 'Chooser' (selector), permitiendo que el usuario elija su aplicación de preferencia para enviar el texto plano generado por la lógica de negocio, mejorando la interoperabilidad de la app con el ecosistema Android



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.